



U N I V E R S I D A D
COMPLUTENSE
M A D R I D

Práctica 1

Introducción al entorno de desarrollo ESP-IDF

Alumnos: Félix García Narocki y Daniel Barroso Corral

Máster Internet de las Cosas

Asignatura: Redes Protocolos e Interfaces I

Objetivo de la práctica.....	3
Ejercicios.....	3
Ejercicio 1.....	3
Ejercicio 2.....	3
Ejercicio 3.....	4
Ejercicio 4.....	4
Ejercicio 5.....	4
Ejercicio 6.....	5
Ejercicio 7.....	6
Ejercicio 8.....	6
Ejercicio 9.....	7

Objetivo de la práctica

El objetivo principal es familiarizarse con el entorno de desarrollo para el ESP32, adquiriendo las competencias necesarias para compilar, flashear y monitorizar proyectos sencillos basados en el framework oficial ESP-IDF. A lo largo de este proceso, se profundiza en el funcionamiento interno de las aplicaciones que aprovechan las capacidades Wi-Fi del microcontrolador, aprendiendo tanto a personalizar las variables de configuración de cada proyecto como a gestionar y responder adecuadamente a los eventos básicos de red.

Temática

Entorno y flujo de trabajo. Inspección hardware del ESP. Multitarea (tasks). Configuración del proyecto. Wi-Fi Scan. Wi-Fi Station y eventos.

Ejercicios

Ejercicio 1

“Modifica el período de suspensión de la tarea para que sea mayor o menor y comprueba que este cambio modifica efectivamente el comportamiento del firmware cargado.”

Para el primer ejercicio hemos modificado el tiempo para que espere el doble de tiempo antes de reiniciar. Esto lo hemos hecho multiplicando por dos el tiempo del vTaskDelay y en el printf para representar el cambio.

Ejercicio 2

Modifica el programa para que únicamente compruebe si el SoC dispone de capacidades WiFi y BLE y muestre la información correspondiente (Sí/No) por la salida estándar. Prueba a ejecutar el programa en los dos tipos de ESP32 de los que disponemos: ESP32 y ESP32-C3.

Para resolver este ejercicio hemos utilizado un condicional terciario en el que comprobamos utilizando chip_info.features y la máscara apropiada si disponemos de WIFI y de BLE. Si resulta que sí, imprime si por pantalla SI y si no, imprime NO.

En los dos esp32 imprime que sí en los dos casos.

Se muestra una captura con el resultado de los dos ejercicios anteriores.

```
Hello world!  
This is esp32c3 chip with 1 CPU core(s), WiFi/BLE, silicon revision v0.4, 2MB external flash  
Minimum free heap size: 340108 bytes  
El dispositivo dispone de WiFi: Sí  
El dispositivo dispone de BLE: Sí  
Restarting in 20 seconds...  
Restarting in 18 seconds...  
Restarting in 16 seconds...
```

```
2 (234) main_task: calling app_main()  
Hello world!  
This is esp32 chip with 2 CPU core(s), WiFi/BTLE, silicon revision v3.0, 2MB external flash  
Minimum free heap size: 304740 bytes  
El dispositivo dispone de WiFi: Sí  
El dispositivo dispone de BLE: Sí  
Restarting in 20 seconds...  
Restarting in 18 seconds...  
Restarting in 16 seconds...  
█
```

Ejercicio 3

Implementa una modificación del programa `hello_world` que implemente y planifique dos tareas independientes con distinta funcionalidad (en este caso, es suficiente con mostrar por pantalla algún mensaje) y distintos tiempos de suspensión. Comprueba que, efectivamente, ambas tareas se ejecutan concurrentemente.

Para esta tarea había que crear dos tareas.

Hemos creado una primera en la que imprime por pantalla en inglés “Hola desde la primera” y otra en la que imprime “Hola desde la segunda”. Estas dos tareas se ejecutan concurrentemente mientras el programa avanza ejecuta el código correspondiente al ejercicio 2.

```
[MAIN] Restarting in 20 seconds...  
[Task1] Hello from task1 in 19 seconds...  
[Task2] Hello from task2 in 19 seconds...  
[Task1] Hello from task1 in 18 seconds...  
[Task2] Hello from task2 in 18 seconds...  
[MAIN] Restarting in 18 seconds...  
[Task1] Hello from task1 in 17 seconds...  
[Task2] Hello from task2 in 17 seconds...  
[Task1] Hello from task1 in 16 seconds...  
[Task2] Hello from task2 in 16 seconds...  
[MAIN] Restarting in 16 seconds...  
[Task1] Hello from task1 in 15 seconds...  
[Task2] Hello from task2 in 15 seconds...  
[Task1] Hello from task1 in 14 seconds...  
[Task2] Hello from task2 in 14 seconds...  
[MAIN] Restarting in 14 seconds...
```

Ejercicio 4

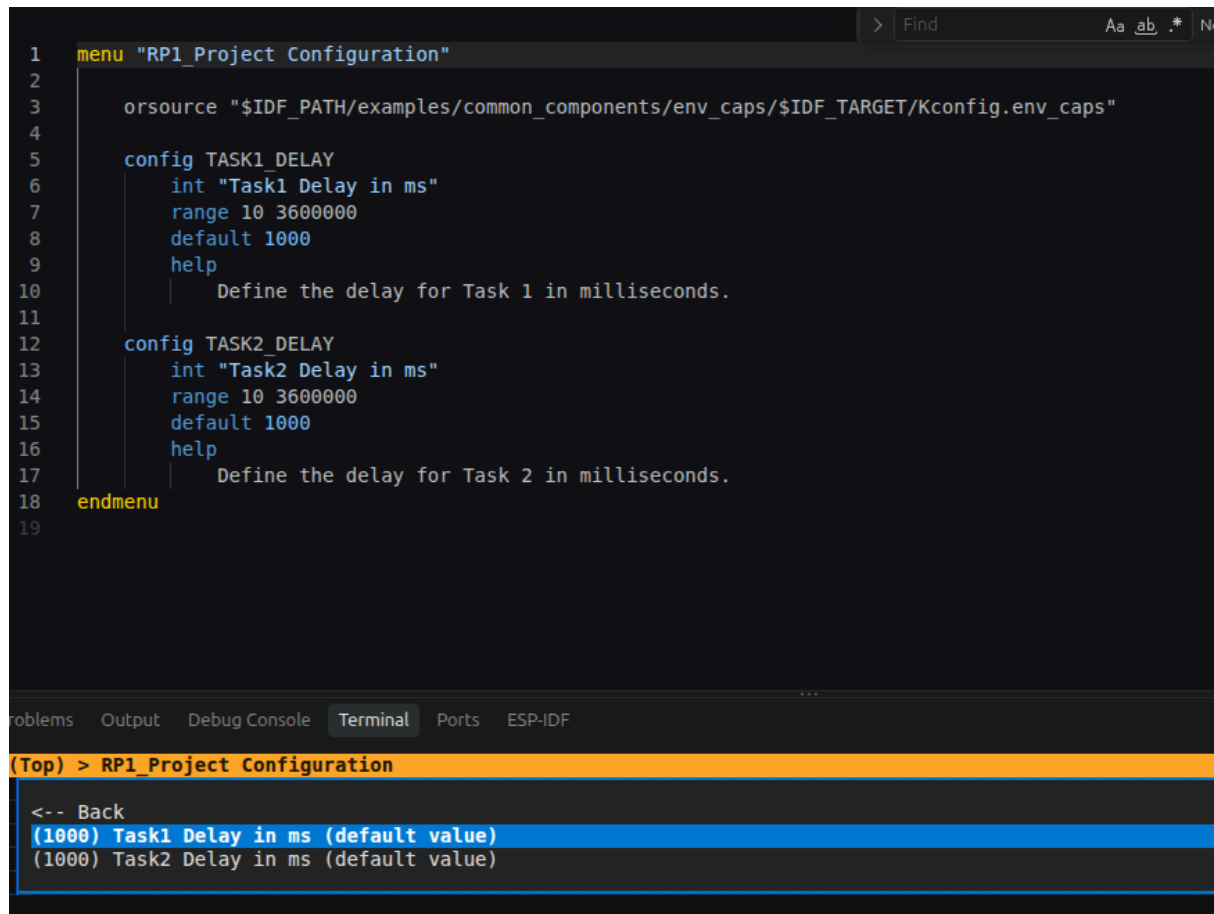
Prueba a ejecutar el proyecto `blink` en un `ESP32-C3` configurando la opción `Blink GPIO number` para usar el pin `GPIO 2`. ¿Qué ocurre?

Al poner el GPIO number en 2 el led se enciende y se apaga.

Ejercicio 5

Modifica el proyecto `hello_world` para que defina dos opciones de configuración que permitan definir el tiempo de espera de cada una de las dos tareas que hayas definido en el ejercicio 3. Haz uso de ellas en tu código y comprueba que su modificación a través del sistema de menús permite una personalización del comportamiento de tu código.

Para realizar este ejercicio hemos abierto el proyecto de blink led y hemos modificado el `Kconfig.projbuild` de la siguiente manera. Permittiéndonos elegir el tiempo de espera de la tarea.



```
1 menu "RP1_Project Configuration"
2
3     orsource "$IDF_PATH/examples/common_components/env_caps/$IDF_TARGET/Kconfig.env_caps"
4
5     config TASK1_DELAY
6         int "Task1 Delay in ms"
7         range 10 3600000
8         default 1000
9         help
10            Define the delay for Task 1 in milliseconds.
11
12     config TASK2_DELAY
13         int "Task2 Delay in ms"
14         range 10 3600000
15         default 1000
16         help
17            Define the delay for Task 2 in milliseconds.
18 endmenu
```

(Top) > RP1_Project Configuration

<-- Back
(1000) Task1 Delay in ms (default value)
(1000) Task2 Delay in ms (default value)

```
[Task1] Delay for 2000 ms
[Task1] Hello from task1 in 40 seconds...
[Task2] Delay for task2 is 3000 ms
[Task2] Hello from task2 in 60 seconds...
El dispositivo dispone de WiFi: Sí
El dispositivo dispone de BLE: Sí
[MAIN] Restarting in 20 seconds...
[Task1] Hello from task1 in 38 seconds...
[MAIN] Restarting in 18 seconds...
[Task2] Hello from task2 in 57 seconds...
[Task1] Hello from task1 in 36 seconds...
[MAIN] Restarting in 16 seconds...
```

Ejercicio 6

Compila, flashea y monitoriza el ejemplo scan situado en el directorio wifi. Crea un nuevo proyecto a partir de este ejemplo y amplía el número máximo de redes a escanear a 20 a través del menú de configuración. Crea un punto de acceso WiFi con tu teléfono móvil y observa que es escaneado por el ESP32.

```
danibc@daniel-ThinkPad-P53:~/Documentos/PracticasMaster/RP1/scan$ idf.py -p /dev/ttyUSB0 flash monitor
I (3226) scan: Total APs scanned = 30, actual AP number ap_info holds = 20
I (3226) scan: SSID      Nothing
I (3226) scan: RSSI      -29
I (3226) scan: Authmode   WIFI_AUTH_WPA2_PSK
I (3226) scan: Pairwise Cipher  WIFI_CIPHER_TYPE_CCMP
I (3236) scan: Group Cipher   WIFI_CIPHER_TYPE_CCMP
I (3236) scan: Channel      1
I (3246) scan: SSID      IoT18
I (3246) scan: RSSI      -51
I (3246) scan: Authmode   WIFI_AUTH_WPA2_PSK
I (3246) scan: Pairwise Cipher  WIFI_CIPHER_TYPE_CCMP
I (3256) scan: Group Cipher   WIFI_CIPHER_TYPE_CCMP
I (3256) scan: Channel      3
I (3266) scan: SSID      iPhone de Cristhian
I (3266) scan: RSSI      -51
I (3266) scan: Authmode   WIFI_AUTH_WPA2_PSK
I (3276) scan: Pairwise Cipher  WIFI_CIPHER_TYPE_CCMP
I (3276) scan: Group Cipher   WIFI_CIPHER_TYPE_CCMP
I (3286) scan: Channel      6
I (3286) scan: SSID      UCM
I (3286) scan: RSSI      -51
I (3286) scan: Authmode   WIFI_AUTH_OPEN
I (3296) scan: Pairwise Cipher  WIFI_CIPHER_TYPE_NONE
I (3296) scan: Group Cipher   WIFI_CIPHER_TYPE_NONE
I (3306) scan: Channel      13
I (3306) scan: SSID      eduroam
I (3306) scan: RSSI      -51
I (3306) scan: Authmode   WIFI_AUTH_ENTERPRISE
```

Ejercicio 7

Analiza por encima el código de la función `wifi_scan` (tarea principal). Céntrate especialmente en las líneas que permiten activar y configurar el escaneado de redes. Intenta comprender el funcionamiento general del programa, prestando especial atención a las funciones con prefijo `esp_wifi_*`. Para más información, por ahora puedes consultar la documentación oficial de ESP-IDF.

Resuelto en discusión con el compañero.

Ejercicio 8

Crea un proyecto a partir del ejemplo `station` situado en el directorio `wifi/getting_started`. Compíllalo, flaséalo y monitoriza su salida estándar. Acuérdate de modificar el SSID de la red a la que se conectará, así como la contraseña elegida a través del sistema de menús de configuración.

Para hacer este ejercicio hemos creado el proyecto `station`. Usando `menuconfig` hemos configurado el SSID y la contraseña y hemos flasheado con el siguiente resultado.

```

I (1938) wifi:state: assoc -> run (0x10)
I (1988) wifi:connected with Nothing, aid = 1, channel 12, BW20, bssid = 52:7d:0f:6c:06:9c
I (1988) wifi:security: WPA2-PSK, phy: bgn, rssi: -39, cipher(pairwise:0x3, group:0x3), pmf:0
I (2008) wifi:pm start, type: 1

I (2008) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (2008) wifi:set rx beacon pti, rx_bcn_pti: 0, bcn_timeout: 25000, mt_pti: 0, mt_time: 10000
I (2028) wifi:dp: 2, bi: 102400, li: 4, scale listen interval from 307200 us to 409600 us
I (2028) wifi:AP's beacon interval = 102400 us, DTIM period = 2
I (3048) esp_netif_handlers: sta ip: 10.164.5.64, mask: 255.255.255.0, gw: 10.164.5.246
I (3048) wifi station: got ip:10.164.5.64
I (3048) wifi station: connected to ap SSID:Nothing password:12345678
I (3048) main_task: Returned from app_main()

```

Ejercicio 9

Modifica el firmware para que el handler (función `event_handler`) encargado del tratamiento de la obtención de una dirección IP sea independiente del tratamiento del resto de eventos del sistema WiFi. Comprueba que, efectivamente sigue observándose la salida asociada a dicho evento, aun cuando ambas funciones sean independientes.

Para este ejercicio hemos creado un nuevo handler para el evento de ip en el que hemos sacado el de ip del `event_handler`.

En un primer momento no ha podido conectar pero en el primer reintento si que lo ha logrado.

```

I (548) wifi station: retry to connect to the AP
I (548) wifi station: connect to the AP fail
I (2838) wifi:state: init -> auth (0xb0)
I (2858) wifi:state: auth -> assoc (0x0)
I (2868) wifi:state: assoc -> run (0x10)
I (2898) wifi:connected with Nothing, aid = 1, channel 1, BW20, bssid = 52:7d:0f:6c:06:9c
I (2898) wifi:security: WPA2-PSK, phy: bgn, rssi: -40, cipher(pairwise:0x3, group:0x3), pmf:0
I (2908) wifi:pm start, type: 1

I (2908) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (2908) wifi:set rx beacon pti, rx_bcn_pti: 0, bcn_timeout: 25000, mt_pti: 0, mt_time: 10000
I (2978) wifi:dp: 2, bi: 102400, li: 4, scale listen interval from 307200 us to 409600 us
I (2978) wifi:AP's beacon interval = 102400 us, DTIM period = 2
I (3938) esp_netif_handlers: sta ip: 10.164.5.64, mask: 255.255.255.0, gw: 10.164.5.246
I (3938) wifi station: got ip:10.164.5.64
I (3938) wifi station: connected to ap SSID:Nothing password:12345678
I (3938) main_task: Returned from app_main()

```